Working document v8, 13 October 2026, the companion to the note "What an Unreliable Verifier Collects" (alatip.github.io/what-an-unreliable-verifier-collects/). Sections 1 to 8 are the plan as written before the experiments ran; Section 9 reports what each found; Section 10 reads the results by hypothesis. Earlier versions are kept locally.

Selection with an Unreliable Verifier on Agent Tasks

Aziz Latipov · working document v8 · terms, what is already known, a formalism, one first measurement, six hypotheses, and a plan with prices

0. The question

This document continues an earlier paper draft of mine, "Exploring the Boundaries of Multi-Agent Ensembles" (v8, 12 October 2026), cited below as the paper. Terms used in this paragraph are defined in Section 1; an **operator** is any rule that turns several candidates into one answer, such as a majority vote or a router that picks one system in advance. The paper's last open result is this. On agent benchmarks, the share of tasks that at least one candidate solves is 10 to 32 points above the best single system. No operator that only rearranges the candidates, such as voting or routing, reaches it. Only a selector that checks each candidate against something outside the solvers could. The paper's bound for such a selector assumes a verifier with one error rate and says nothing about a verifier that is itself unreliable. This document collects what is known about that case, writes a small formalism for it, tests the formalism once on public data at zero cost, and lists what to measure next and at what price.

1. Terms, in the order they are needed

Task, candidate, trajectory, hidden test

A **task** is one benchmark item: a GitHub issue on SWE-bench Verified, a customer conversation on τ-bench, a terminal job on Terminal-Bench. A **candidate** is one full attempt at the task by an agent: a patch, a finished conversation, a changed file system. The full record of the attempt is its **trajectory**; a candidate is also called a rollout or a trial. The benchmark decides whether a candidate is **correct** with a check the agent never sees: the **hidden tests** on SWE-bench, a database-state and conversation check on τ-bench. Throughout, "correct" means "passes the benchmark's hidden check".

n*, k_i, p_i, *N

For task i there are n candidates, of which k_i are correct. $p_i = k_i/n$ is the task's **base rate**, the share of candidates that are correct. A task with a small p_i is **hard**, in the only sense used here. N is the number of tasks in the benchmark.

pass@1, pass@*n* (the oracle), best member, prize, contested task

pass@1 is the accuracy of one candidate, the mean of p_i over tasks. **pass@*n***, also called the **oracle**, is the share of tasks with $k_i \geq 1$, which is what a perfect selector would reach. When the candidates come from different systems, as on a leaderboard, the comparison point is the **best member**, the single system with the highest accuracy, instead of pass@1. The **prize** is the oracle minus pass@1 (or minus the best member), in percentage points. A task is **contested** when $0 < k_i < n$. The prize lives entirely on contested tasks.

Share collected

For any selection rule,

$$\text{share} = [\text{acc(rule)} - \text{pass@1}] \,/\, [\text{oracle} - \text{pass@1}]$$

(how much of the prize the rule collects: 1 is the oracle, 0 is a random candidate, negative is worse than random)

Band, top seven

On the SWE-bench Verified leaderboard a **band** is the set of submissions whose accuracy lies in one interval of width 0.1, such as 0.6 to 0.7. The **top seven** are the seven most accurate submissions.

φ, n_{eff}, one difficulty factor

φ is the Pearson correlation between two candidates' correctness vectors over tasks. **Kish's effective number** turns the mean φ of n candidates into the number of independent candidates they are worth:

$$n_{\text{eff}} = n \,/\, [1 + (n - 1) \cdot \varphi]$$

(n candidates with φ = 0 are worth n; with φ = 1 they are worth one)

The paper's main empirical fact is that candidates from different models and from repeated runs fail on the same tasks, and that one latent variable, task difficulty, accounts for almost all of this shared failure. The n_{eff} of the top seven SWE-bench submissions is 1.38; four τ-bench trials of one agent are worth 1.37 to 1.82. The consequence for this document is that the prize sits on tasks with small p_i.

Verifier, execution-based, execution-free

A **verifier** is any procedure that looks at a candidate and says accept or reject, or gives a score that is then thresholded or ranked. An **execution-based** verifier runs something: tests the

agent wrote, tests generated by another model, the repository's own regression tests. An **execution-free** verifier, also called a model verifier, is a language model that reads the patch or trajectory and estimates correctness; a trained one is called a critic or reward model, an untrained one a judge. The hidden test is a verifier with no error. Every other verifier has two error rates.

α, β, operating point (examples of operating points from practice at the end of this section)

α (false accept) is the probability that the verifier accepts a wrong candidate. **β** (false reject) is the probability that it rejects a correct one. A threshold on a verifier's score fixes one (α, β) pair, called an **operating point**; a stricter threshold lowers α and raises β. The curve of all operating points is the ROC curve, and **AUROC**, the area under it, is the probability that a random correct candidate scores above a random wrong one (0.5 is chance).

Precision on a task

Among the candidates the verifier accepts on task i, the share that are correct:

$$\text{precision}_i = [p_i \cdot (1 - \beta)] / [p_i \cdot (1 - \beta) + (1 - p_i) \cdot \alpha]$$

(accepted correct candidates divided by all accepted candidates; it depends on the task's base rate, not only on the verifier)

Selection rules

Threshold rule: accept every candidate above a threshold, pick one of the accepted at random, and if none is accepted pick any candidate at random. **Argmax rule**: pick the candidate with the highest score. **Hybrid rule**: one verifier narrows the field to m candidates, a second chooses among them.

i.i.d. verdicts, per-task verdicts, ICC

In the simplest model every verdict is an independent coin with the same α and β on every task (**i.i.d. verdicts**). In the **per-task** model each task draws its own pair (α_i, β_i) from a distribution, so the verifier is systematically lenient on some tasks and systematically strict on others, and verdicts within a task are correlated. The **intraclass correlation** (ICC) of verdicts within a task measures the distance from i.i.d.: 0 means independent coins, 1 means every candidate of a task gets the same verdict.

In-sample, split-half

A quantity is estimated **in-sample** when it is fitted on the same candidates it is then used to predict. A **split-half** estimate fits on one random half of each task's candidates and predicts the other half.

Examples of operating points from practice

Three groups of verifiers, by which of the two errors is non-zero. The candidate is an answer, a patch or a proof; the verifier is whatever checks it.

(a) α > 0, β ≈ 0: a lenient verifier, lets bad through, never kills good.

- A compiler or type checker as verifier of a patch: a correct patch always compiles (β = 0), and so does almost any wrong one (α near 1).
- The repository's regression tests for a bug: the right fix passes them, and so does a wrong fix that breaks nothing. In the data of Section 4 their AUROC is 0.52, nearly a coin.
- Signature-based antivirus: it almost never flags a clean file, and misses new or packed malware.
- The check "the answer is a number" on a mathematics problem: the right answer passes, and so does any wrong number.

(b) β > 0, α ≈ 0: a strict verifier, lets nothing bad through, kills some good.

- Exact string match against a reference answer: a wrong answer never passes, a right one in another form ("1/2" for "0.5") is rejected. This is the "strict format" of Section 2.1: 0 of 54 false accepts at recall 0.68.
- Over-strict or flaky tests for a patch: if the test itself fails on the reference fix (in Agentless, 213 of 300 reproduction tests failed on the bug but only 94 also passed on the correct patch), it rejects correct patches without accepting wrong ones.
- Formal proof checking in Lean or Coq: a wrong proof never passes; a paper-correct argument that could not be formalised, or lacks a library lemma, is rejected.
- A strict WAF or allow-list: it blocks the attack and also blocks unusual legitimate requests.

(c) α > 0 and β > 0: both errors non-zero, and the threshold trades one for the other.

- A language model judging a patch or proof: it accepts a convincingly written wrong one (α) and rejects a correct one that is unusually formatted or on a hard task (β). This is the verifier of Section 4: threshold 0.5 gives α 0.31 and β 0.23, threshold 0.7 gives α 0.05 and β 0.76.
- Model-generated tests: a weak test passes a wrong patch (α), a test that misread the specification rejects a correct one (β). On HumanEval such tests have precision 0.88 and recall 0.70 (VALTEST, 2411.08254).
- A static vulnerability analyser: it misses vulnerabilities it does not know and flags safe code; its severity scale is the threshold.
- Human code review under a deadline: misses a bug in a large diff, rejects a correct solution it did not understand.
- Counterexample search by enumeration: finds none up to the search bound and "accepts" a false conjecture (α); finds a spurious "counterexample" through a bug in the search and

rejects a true one (β).

Two observations from this list that hypotheses H2 and H4 rest on. Execution-based checks (tests, Lean, exact match) sit by nature nearer group (b): they rarely accept the wrong thing but often kill the right one through form or their own fragility. Model and human checks sit in group (c), and their errors grow where the task is hard for them too.

With these terms, Section 2 states what is already measured, Section 3 the formalism, Section 4 the one measurement made so far, Section 5 the hypotheses and Section 6 the plan.

2. What is already known

2.1 From the paper

This subsection restates the paper's results that this document builds on; nothing here is new.

- **The prize on agents.** SWE-bench Verified, 175 submissions on 500 bugs. Top seven: best member 0.792, oracle 0.890, prize 9.8 points. Bands 0.6–0.7, 0.5–0.6 and 0.4–0.5 (31, 31 and 28 submissions): prize 18.8, 25.6 and 32.2 points. τ-bench, one agent run four times on the same tasks: pass@4 exceeds pass@1 by 14 to 25 points.
- **Rearranging collects nothing.** The majority vote of the top seven collects 0.02 of its prize. In the bands the majority ties or trails the best member. A majority of four τ-bench trials is not defined without a tie rule: counting a 2-of-4 split as a pass or as a fail moves the result by about ten points.
- **Why: one difficulty factor.** φ among the top seven is 0.68, n_{eff} 1.38; a model with that one latent difficulty variable reproduces their joint failures.
- **Model judges as certifiers, measured on text problems.** The paper had five language models judge 54 wrong answers to text problems (mathematics and multiple-choice science) and say whether each answer was correct. The judges' false accepts correlate at φ 0.38 (n_{eff} 2.0 of 5). Only unanimity of all five keeps false accepts low: 1 of 54 wrong answers passes, while 0.85 of correct answers are still accepted (its recall; 0 of 54 and recall 0.68 when an answer in the wrong format counts as wrong). Unanimity of the three weakest judges passes 8 of 54. Judges accept their own wrong answers at rates from 0.36 to 1.00. This is α for model judges, measured on text; it is large.
- **The paper's bound.** With $P_{\text{all wrong}} = 1 -$ oracle (the share of tasks on which every candidate is wrong, so there is nothing to select) and ε the verifier's total error rate, the paper states

$$\text{acc(selection)} \le (1 - P_{\text{all wrong}}) \cdot (1 - \varepsilon)$$

(acc(selection) is the selector's accuracy, the share of tasks on which the candidate it picks is correct, the same as acc(rule) in Section 1; the selector cannot do better than the oracle times the verifier's success rate). This is our own claim from the paper, and it is so far untested: no real verifier has been put into it. It merges α with β, and it gives every task the same ε.

2.2 From the literature on agents

Across the 22 published systems reviewed for this document (working file LIT-agents.md), selectors with a verifier collect between 0.23 and 0.63 of the prize, about half of them between 0.3 and 0.55, and no paper reports the verifier's α and β against the hidden tests. Table 2.1 keeps the rows where pass@1, the oracle and the selected accuracy are all reported, so the share can be computed.

Table 2.1. Best-of-*n* with a verifier on agent benchmarks. Share = (selected – pass@1) / (oracle – pass@1), computed here from the papers' numbers; TB2 = Terminal-Bench 2.

System (arXiv id)	Benchmark	*n*	pass@1	Oracle	Verifier	Selected	Share
CodeMonkeys (2501.14723)	SWE-bench Verified	10×8	45.8 (random pick)	69.8	generated tests, vote, then model choice	57.4	0.48
SWE-Gym (2412.21139)	SWE-bench Verified	8 / 16	20.6	37.8 / 42.8	trained critic, 32B	29.8 / 32.0	0.53 / 0.51
R2E-Gym (2504.07164)	SWE-bench Verified	26	34.4	64.4	tests / model / hybrid	43.7 / 42.8 / 51.0	0.31 / 0.28 / 0.55
Nemotron-Cascade (2512.13607)	SWE-bench Verified	32	37.2	57.7	regression tests, then reproduction tests	43.6	0.31
Agentic Rubrics (2601.04171)	SWE-bench Verified	16	22.6 / 39.6 (random pick)	51.4 / 65.6	rubric judged by GPT-5 (AUROC 0.886)	40.6 / 54.2	0.63 / 0.56

SWE-Hero (2604.01496)	SWE-bench Verified	32	50.0 / 57.4 / 60.1	76.0 / 78.4 / 79.8	four trained critics, best of them	57.9 / 62.6 / 64.6	0.30 / 0.25 / 0.23
Kim et al. (2604.16529)	SWE-bench Verified; TB2	16	70.9; 47.0	85.4; 70.5	summaries, tournament of model votes	77.6; 59.1	0.46; 0.52
LLM-as-a-Verifier (2607.05391)	SWE-bench Verified	3	76.1	84.4	pairwise model tournament	78.2	0.25
Agent S3 (2510.02250)	OSWorld	4–10	62.6	74.7–80.5	model judge of narratives	69.9–72.6	0.56–0.60 (pool pass@1 not reported)
Zhu et al. (2506.12928)	GAIA	4	55.8	69.1	list-wise model verifier	63.0	0.54

Reading the table. Four things are visible without further data. First, the prize on agents is large everywhere, 8 to 30 points between pass@1 and the oracle, and no selector closes it; the best collect about 0.6, the typical ones a third. Second, what the verifier checks matters more than how large it is: generated tests alone (R2E, Nemotron-Cascade) give 0.31, a model verifier alone 0.28, and their hybrid on the same candidates 0.55; a rubric read by GPT-5 gives 0.56 to 0.63. Third, the stronger the generator, the smaller the share the same kind of verifier collects: SWE-Hero 0.30 at pass@1 50 and 0.23 at 60, LLM-as-a-Verifier 0.25 at 76. This agrees with consequence (c) of Section 3: in a strong pool the remaining contested tasks are the hardest, and there the verifier errs most. Fourth, the shares are measured at different n, from 3 to 32, and the rows cannot be compared directly: by Section 3 the share falls with n, so 0.25 at n=3 and 0.23 at n=32 are not the same verifier quality. What the table lacks: a single row with α and β, so it cannot say which of the two errors eats two thirds of the prize.

Three further facts from the same review. Trae Agent (ByteDance, 2025), Agent S3 and the General AgentBench study (2602.18998) each report that a model selector's accuracy peaks at four to six candidates and falls beyond; SWE-Hero reports that the distance from the selected accuracy to the oracle widens as n grows. Among 27 vision-language judges of computer-use trajectories, over-accepting makes up about two thirds of all judge errors (OSReward, 2607.28609). Of the 22 systems, only SWE-RM (2512.21919) reports the verifier's AUROC on SWE-bench, 0.78; CodeMonkeys is the only one to price selection, at 5.8% of its total spend. On τ-bench no published work selects among full trajectories with a verifier.

2.3 From the theory

The theory covers independent candidates on one task and a verifier described by its ROC curve. Its main result: as n grows, the accuracy of selection tends not to the oracle but to the verifier's precision at the task's base rate, and reaches 1 only if some operating point has α exactly 0 (Dorner et al., 2507.12399; Stroebl, Kapoor and Narayanan, 2411.17501). Second, picking the argmax of a noisy score can get worse as n grows: the accuracy lost relative to the best candidate grows like the square root of n times the squared score error (Huang et al., 2503.21878). A threshold rule at equal compute is never worse than argmax and does not degrade (2507.12399). Third, when one candidate is passed through several checks whose false-accept rate varies by task, the per-task variation makes the checks far less effective than independence would predict (Han, 2607.13918). Empirically, a model's self-verification fails largely where its generation fails: the odds ratio is about 57 (2605.10850).

To avoid the impression that everything is settled: what is proven above concerns one task with a fixed base rate and independent candidates. The following is neither proven nor computed by anyone. First, what selection yields on a whole benchmark, where candidates are correlated through difficulty and the distribution of k_i has many tasks at or near zero. Second, the joint effect of α and β at finite n on real k_i, including their asymmetry. Third, verifier error rates that vary by task inside a selection among n candidates (Han treats one candidate; 2605.10850 is empirical only). Fourth, any measurement of a verifier's α and β on agent tasks. The formalism of Section 3 glues the known pieces into one formula and runs it on real k_i; the only new part is the per-task version. The main new thing in this document is not the theory but the measurement in Section 4.

3. The formalism

This section states the model, derives three consequences in words, and applies the model to the paper's SWE-bench and τ-bench matrices with assumed (α, β) to see what shares it predicts.

3.1 The model

Symbols, as in Section 1: n candidates on one task, k of them correct; α and β the verifier's false-accept and false-reject rates; N tasks. Two new ones: A_r is the number of correct candidates the verifier accepts, A_w the number of wrong ones it accepts. The rule is the threshold rule: pick uniformly among accepted candidates, or uniformly among all n if none is accepted.

$$A_\mathrm{r} \sim \mathrm{Binomial}(k,\ 1 - \beta)$$

(how many of the k correct candidates survive the verifier, each with chance $1 - \beta$)

$A_w \sim \text{Binomial}(n - k, \alpha)$

(how many of the $n - k$ wrong candidates slip through, each with chance α)

$\text{acc}(k, n, \alpha, \beta) = \text{E}[\, A_r / (A_r + A_w)\,]$

(the chance that the candidate picked from the accepted pool is correct; when nothing is accepted the ratio is replaced by k/n; the expectation is computed exactly by summing over all values of A_r and A_w)

$\text{acc(selector)} = (1/N) \cdot \Sigma_i \, \text{acc}(k_i, n, \alpha, \beta)$

(the average over the N tasks, each with its own observed k_i; the shared failures of Section 1 enter here, not as an assumption but through the observed distribution of k_i, which has many tasks at or near zero)

The per-task version replaces (α, β) by (α_i, β_i) inside the sum, one pair per task.

3.2 Three consequences

(a) A precision ceiling. Let $p = k/n$ be the task's base rate and let n grow with p fixed.

$\text{E}[A_r] = n \cdot p \cdot (1 - \beta)$

(expected number of accepted correct candidates)

$\text{E}[A_w] = n \cdot (1 - p) \cdot \alpha$

(expected number of accepted wrong candidates; both grow in proportion to n, so their ratio does not)

$\text{acc}(k, n, \alpha, \beta) \to \text{precision}(p) = [p \cdot (1 - \beta)] / [p \cdot (1 - \beta) + (1 - p) \cdot \alpha]$

(what the selector tends to on the task as n grows: the verifier's precision, not 1. Why: by the law of large numbers $A_r/n \to p \cdot (1 - \beta)$ and $A_w/n \to (1 - p) \cdot \alpha$; the ratio $A_r/(A_r + A_w)$ is a continuous function of these two shares with a denominator bounded away from zero, so it converges to the ratio of the limits; it is bounded by 1, so its expectation converges too, and the probability that nothing is accepted goes to zero. Assumptions: verdicts independent given the task, p fixed as n grows. The same is equation (5) of Dorner et al., 2507.12399)

$\text{precision}(p) \approx [p \cdot (1 - \beta)] / \alpha$ when p is small and $p \cdot (1 - \beta) \ll \alpha$

(on a hard task the accepted pool is mostly wrong candidates that slipped through)

The prize is concentrated on exactly these tasks. On the SWE-bench bands, of the bugs the best member fails and some other member solves, 45 of 94 (band 0.6–0.7), 63 of 128 (0.5–0.6) and 89

of 161 (0.4–0.5) have $p_i \le 0.2$ (counts rounded from the shares 0.48, 0.49 and 0.55). So α is paid where the money is.

(b) α and β are not symmetric. Under i.i.d. verdicts a false reject loses the task only if every correct candidate is rejected, with probability β^k; more candidates heal it. A false accept adds wrong candidates to the accepted pool in proportion to $n - k$; more candidates hurt. So for a given verifier there is an optimal n. A verifier with small α and large β, such as a strict but flaky test, should gain with n; one with large α and small β, such as a lenient judge, should not. Under per-task rates the loss from false rejects is $E[\beta_i^k]$, which is larger, and that is why consequence (c) matters.

(c) Per-task error rates matter more than their mean. If the verifier is lenient on some tasks and strict on others, the mean (α, β) overstates what it collects. On tasks where it is lenient it accepts every wrong candidate; on tasks where it is strict it rejects every correct one. Neither is fixed by sampling more candidates.

3.3 What the formula predicts on the paper's matrices

The formula was applied with assumed (α, β) to real k_i: on SWE-bench, random panels of n submissions from the band 0.6–0.7 (60 panels per cell); on τ-bench, the fifteen runs of the paper's τ-bench trial logs (four models on the airline, retail and telecom domains; thirteen runs with four trials per task, two with eight), each run's own trials being the candidates. Nothing is measured here; the point is what the formula says before any verifier is seen.

Table 3.1. Predicted share of the prize under the threshold rule. SWE-bench: band 0.6–0.7. τ-bench: median of the thirteen four-trial runs and mean of the two eight-trial runs. The oracle used for the share is the panel's or run's own pass@*n*.

(α, β)	SWE-bench, *n*=4	SWE-bench, *n*=8	τ-bench, *n*=4	τ-bench, *n*=8
(0.05, 0)	0.91	0.87	0.92	0.89
(0.10, 0)	0.82	0.75	0.84	0.79
(0.20, 0)	0.66	0.54	0.70	0.62
(0.30, 0)	0.51	0.39	0.58	0.49
(0, 0.30)	0.80	0.88	0.80	0.90
(0, 0.60)	0.47	0.66	n/a	n/a
(0.20, 0.30)	0.44	0.38	0.51	0.49

Three readings. First, the columns differ by at most 0.11 across the two benchmarks: the share is mostly a function of (α, β, n), not of the data. At n=4 and β=0, for α up to 0.3, the share is approximately 1 − 1.5α (the SWE-bench slope is nearer 1.6). Second, the asymmetry of consequence (b) shows: raising β from 0 to 0.3 costs 20 points of share at n=4 and 12 at n=8; raising α from 0 to 0.3 costs 49 and 61 (even α=0.2 costs 34 and 46). Third, consequence (c): with α=0.2 and n=4, a verifier whose $\alpha_i = c \cdot (1 - p_i)$, with c chosen so the mean over tasks is 0.2, would collect 0.32 instead of 0.67 among the top seven, where the prize sits on the few hardest bugs, and 0.65 instead of 0.66 in the band 0.4–0.5, where difficulty is spread. Under the formula even a sloppy verifier with α=0.3 would still add about 9 points over the best member in the band 0.4–0.5 at n=8 (8.3 if its errors track difficulty), and 1.3 points among the top seven at n=4 with difficulty-tracking errors (4.4 if independent). These are predictions from assumed rates; the next section measures the rates for one real verifier and checks the formula against observed selection.

4. One measurement: R2E-Gym candidates on SWE-bench Verified

This section measures α, β and their task-dependence for one public model verifier, reports what it collects as a function of n, and checks the formula of Section 3 against the observed selection accuracy. Nothing here was pre-registered; it is a first look, and Section 5 treats its findings as hypotheses for the other data sets, not as results.

4.1 Data

The R2E-Gym authors (2504.07164) released, for their 32B agent, the hidden-test outcome of every attempt on each of the 500 SWE-bench Verified bugs: 12,695 candidates, 19 to 26 per bug (mean 25.4). Each candidate carries the probability of "yes" from the authors' execution-free verifier (a 14B model that reads the patch and trajectory). Each candidate also carries two execution-based numbers, a mean score on tests generated by a test-writing model and the number of repository regression tests passed, and the number of agent steps. The generated-test score is identical across all candidates on 68% of bugs, so it behaves as a per-bug quantity, not a per-candidate verifier, and only the model verifier is treated as a verifier below. The dataset (HF R2E-Gym/32B_predictions, 190 kB) carries no licence statement; it is cited through the paper.

pass@1 is 0.319 and pass@n is 0.644, a prize of 32.5 points. 178 bugs are solved by no candidate, 23 by every candidate, 299 are contested. On contested bugs the base rate p_i has quartiles 0.12, 0.42 and 0.77.

4.2 The verifier's errors

Table 4.1. The model verifier against hidden tests, pooled over 12,695 candidates. AUROC: model verifier 0.805; generated-test score 0.603; regression tests 0.52; fewer agent steps 0.735.

Threshold on P(yes)	α (accepts a wrong candidate)	β (rejects a correct one)	Share of candidates accepted
0.3	0.54	0.06	0.67
0.5	0.31	0.23	0.46
0.6	0.14	0.49	0.26
0.7	0.05	0.76	0.11
0.8	0.003	0.99	0.01

No threshold gives both rates low. Verdicts are clustered by bug: the ICC of accept verdicts within a bug is 0.26 to 0.33 among wrong candidates and 0.27 to 0.43 among correct ones, across the thresholds 0.5 to 0.7. The clustering has a direction. β_i rises with difficulty: its correlation with p_i is −0.50, and at threshold 0.6 it is 0.77 on bugs with $p_i \le 0.2$ against 0.46 on bugs with $p_i > 0.5$. α_i does not: its correlation with p_i is −0.03 at threshold 0.5 and +0.01 at 0.7. On this data the verifier's blind spot is rejecting correct patches on hard bugs, not accepting wrong ones.

4.3 What it collects, as a function of *n*

Table 4.2. Accuracy of selection among *n* random candidates per bug (100 random subsets), with the share of the prize in brackets. "Hybrid" is the R2E-Gym authors' own rule: model verifier narrows to the top three, the generated-test score picks among them.

Rule	*n*=2	4	8	16	25
oracle, pass@*n*	0.407	0.485	0.552	0.612	0.643
random candidate	0.319	0.317	0.319	0.321	0.318
model verifier, argmax	0.362 (0.49)	0.391 (0.44)	0.406 (0.37)	0.421 (0.34)	0.429 (0.34)
hybrid	0.348 (0.32)	0.388 (0.42)	0.418 (0.43)	0.434 (0.39)	0.446 (0.39)
generated-test score, argmax	0.349 (0.33)	0.368 (0.31)	0.382 (0.27)	0.387 (0.23)	0.382 (0.20)
model verifier, threshold 0.5	0.349 (0.34)	0.371 (0.32)	0.381 (0.27)	0.384 (0.22)	0.385 (0.21)

Accuracy rises with n while the share falls, from about a half at two candidates to a third at sixteen, so the distance to the oracle widens. This is consistent with consequence (a), which was

derived for the threshold rule; for the argmax rule the theory of Section 2.3 predicts the same direction. The hybrid is the best rule from eight candidates on and still collects under 0.45.

4.4 The formula against the data

This subsection asks whether the formula of Section 3.1 reproduces the observed accuracy of the threshold rule, first with one pooled (α, β), then with per-bug rates estimated in-sample, then with per-bug rates estimated split-half.

Table 4.3. Threshold rule: observed accuracy against the formula. "Pooled" uses the (α, β) of Table 4.1 (i.i.d. verdicts). "Per-bug, in-sample" estimates (α_i, β_i) from all of a bug's candidates and predicts the same candidates. "Per-bug, split-half" estimates them on a random half of each bug's candidates (each rate pulled toward the pooled one by the weight of one extra candidate, to stop a half with one or two candidates from giving 0 or 1) and predicts the other half; the observed column of that comparison is also on the other half (100 splits). The split-half rows use *n* up to 12 because each half has about 13 candidates.

Threshold	*n*	Observed	Pooled	Per-bug, in-sample	Observed (half)	Pooled (half)	Per-bug, split-half
0.6	4	0.358	0.387	0.359	0.358	0.388	0.370
0.6	8	0.377	0.417	0.377	0.379	0.416	0.396
0.6	12 / 16	0.397 (16)	0.429 (16)	0.392 (16)	0.389 (12)	0.426 (12)	0.412 (12)
0.7	8	0.354	0.398	0.352	0.355	0.400	0.369
0.7	12 / 16	0.369 (16)	0.434 (16)	0.365 (16)	0.363 (12)	0.421 (12)	0.384 (12)

What this shows. The pooled formula overpredicts the observed accuracy by 3 to 7 points; at threshold 0.7 the gap grows with n (4.4 at n=8, 6.5 at 16), at 0.6 it stays at 3 to 4 points from n=4 on. I.i.d. verdicts are the wrong model. Per-bug rates fitted in-sample reproduce the observation within half a point, but with two free parameters per bug that fit is expected and is not evidence. Per-bug rates estimated split-half remove between 38% and 69% of the pooled formula's error (54% at threshold 0.6 and n=8; 64% at 0.7 and n=12), with about 13 candidates per half to estimate each rate. So on this data the verifier's errors are task-specific to a degree that the pooled (α, β) misses, and part of the remaining error is either estimation noise in the per-bug rates or structure the per-task model does not capture. Which of the two is the question E1 in Section 6 answers (answered in Section 9.1: estimation noise).

5. Hypotheses

Each hypothesis is stated with what would falsify it. H1, H3 and H4 were written after the R2E look of Section 4 and are consistent with it; they are predictions for the other data sets, to be pre-registered before E2 to E5 run. H2, H5 and H6 are not yet tested on anything.

1. **H1, precision ceiling.** With any $\alpha > 0$ the share collected falls with n under argmax and threshold rules, and the selector's accuracy converges to the mean per-task precision, not to the oracle. Falsified if on a data set with $\alpha \geq 0.05$ the share is flat or rising from n=4 to the largest n available (16 where there are 16 candidates, 5 on Terminal-Bench). On R2E the share fell from 0.44 at n=4 to 0.34 at n=16 (0.49 at n=2). [Revised after E1, Section 9.1: holds for argmax and lenient thresholds; at strict thresholds with β above 0.5 the share rises up to n=25, as consequence (b) predicts.]
2. **H2, asymmetry.** Execution-based verifiers, which have small α and large β, gain share with n; model verifiers, with larger α, plateau or lose share. The formula's version, that a verifier at (α 0.05, β 0.6) collects more at $n \geq 8$ than one at (α 0.2, β 0.1), is a prediction of the formula not yet tabulated in Table 3.1; E1 adds those cells [Section 9.1: the crossing is between n=8 and 16, so read 'by n=16'], and the data check is to bring two verifiers to those operating points by threshold and compare their shares. Falsified if a per-candidate test verifier's share declines with n like the model verifier's. R2E cannot test it: its test signal is per-bug.
3. **H3, per-task rates.** Verifier verdicts within a task have ICC above 0.2; the pooled formula overpredicts accuracy by 3 points or more at the largest n; per-task rates estimated split-half remove more than a third of that error. Falsified if the pooled formula fits within 1 point, or if split-half per-task rates remove less than a third of its error. R2E: ICC 0.26 to 0.43, overprediction 3 to 7 points, split-half removal 38% to 69%; the threshold of one third is set from these numbers.
4. **H4, where the blind spot is.** For model verifiers the task-dependence is on the reject side: β_i rises with difficulty, α_i does not. For test-based verifiers it is on the accept side: weak generated tests pass wrong patches on hard bugs. Falsified by a model verifier whose α_i correlates with difficulty at $|r| \geq 0.3$, or a test verifier whose α_i does not. R2E model verifier: $r = -0.50$ for β_i, about 0 for α_i.
5. **H5, independence of the verifier.** Two verifiers at the same pooled (α, β) collect different shares: the one whose per-task errors are uncorrelated with the solver's difficulty collects more, and the difference grows with the pool's strength (Section 3.3: 0.67 against 0.32 among the top seven under the formula). Falsified if a verifier of another kind or family

and a same-family judge, matched on (α, β) by threshold, collect the same share within a resampling (bootstrap) interval over tasks. Needs two verifiers on identical candidates.

6. **H6, cost.** Let c_{gen} be the price of one candidate and c_{ver} the price of verifying one. With measured per-task rates, the money-optimal n for a model verifier lies between 4 and 8: beyond it, the gain per added candidate falls below (c_{gen} + c_{ver}) divided by the value of a solved task, at the prices published by the papers of Section 2.2 (CodeMonkeys: verification 5.8% of spend). Falsified if at n from 8 to 16 the marginal gain still exceeds the marginal cost at those prices. This document gives no prices yet; E1 adds them.

6. How to test them, and what it costs

Table 6.1. Experiments in the proposed order. "$0" means no model calls; days are working days of compute and analysis.

#	What	Data	Tests	Cost
E1	Finish R2E, pre-registered: bootstrap intervals on every share; split-half per-bug rates with the two sources of remaining error separated (estimation noise, by simulation at the same candidate counts, against model misfit); (α_i, β_i) against 1 – p_i and against the paper's latent difficulty per bug (an item-response-theory estimate that uses all 175 submissions, not only these candidates); optimal threshold per n; the hybrid rule; a cost curve with candidate and verifier prices from the papers.	R2E-Gym/32B_predictions, on disk	H1, H3, H4 (reject side), H6	$0, 1 day
E2	Same tables on a non-SWE agent benchmark:	github.com/llm-as-a-verifier/llm-as-a-verifier, MIT	H1 ($n \le 5$), H3; judge α	$0, 1 day

	one model (DeepSeek V4 Flash in mini-SWE-agent) run five times per task on Terminal-Bench 2.1, the benchmark's own pass/fail recorded in every trajectory file, and the authors' model-judge scores stored in the repository, so no judge calls are needed. Also Terminal-Bench 4.0 (66 tasks × 5 trials). With five candidates H1 can only be read from n=2 to 5.		and β on a second benchmark	
E3	A per-candidate execution-based verifier: 67,074 rollouts of one model (Qwen3-Coder-480B in OpenHands) on 3,792 real issues, about 18 per issue, each with the benchmark's resolved flag and, for roughly half the rows, two columns: whether the agent's own generated tests fail on the original and pass on the reference fix, and whether the agent's patch passes its own tests. Only those columns need to be read, not the trajectories.	HF nebius/SWE-rebench-openhands-trajectories, CC-BY-4.0, tens of GB in full	H2, H4 (accept side)	$0, 2 days, disk
E4	Five o1 rollouts per SWE-bench Verified bug with the submitter's cross-check matrix	Public AWS bucket swe-bench-submissions, key 20250117_wandb_programmer_o1_crosscheck5, 425 MB	H2 with hidden-test labels; test	$0 in calls, 1 to 2 days of

	(each rollout's patch run against tests generated in the other rollouts), uploaded to the leaderboard's public bucket. Per-rollout correctness must be recomputed with the SWE-bench Docker harness. Run only if E3's test columns are too sparse.		α and β per candidate	local Docker
E5	Two verifiers of different kind on the same candidates, run by us: a cheap judge (Claude Haiku) and a test-writer from another model family, on 200 issues × 8 rollouts from E3. Match their (α, β) by threshold, then compare shares. This repeats on agent tasks the paper's comparison of same-family and cross-family judges on text.	E3 subset	H5 directly; H2 and H4 jointly	$20 to $50, 2 days
E6	Write-up: in the paper, replace the bound of Section 2.1 by the per-task formula; add the *n*-decline and the asymmetry; a short note with the results of E1 to E4, and E5 if run.	n/a	n/a	$0, 2 days

Order: E1 and E2 first, both free, both finishing the formula check on two benchmarks. E3 next, because it is the only free per-candidate test signal and H2 cannot be tested without one. E4 only if E3 falls short. E5 is the one paid experiment and the only one that tests H5. Total: about a week and at most $50. Without E5, H5 stays untested.

7. Caveats

- R2E is one agent, one verifier, one benchmark; its generated-test columns are per-bug, not per-candidate; the data set has no licence statement.
- The threshold rule with a uniform pick among accepted candidates is the cleanest for the formula. Real systems use argmax or hybrids, which the formula does not cover and which the theory says can be worse at large n.
- The per-bug in-sample fit of Table 4.3 is not evidence; the split-half fit is, and it is partial.
- Hidden tests are themselves imperfect (2602.04254); "correct" here means "passes the benchmark's tests".
- The $1 - 1.5\alpha$ rule holds for n=4, β=0, $\alpha \leq 0.3$, one SWE-bench band and the τ-bench runs; it is a reading of Table 3.1, not a derivation.

8. Summary

One public verifier on SWE-bench Verified has AUROC 0.805, no operating point with both error rates low, verdicts clustered by bug, and a blind spot on the reject side that grows with difficulty; it collects half the prize at two candidates and a third at sixteen. A formula with task-specific false-accept and false-reject rates fits the observed selection better than one pooled pair of rates with independent verdicts, and split-half estimates confirm part of that improvement; the paper's single error rate is coarser than either. Four free experiments and one at under $50 would say whether the same holds on a second benchmark, whether execution-based verifiers scale with the number of candidates where model verifiers do not, and whether a verifier independent of the solver collects more at equal error rates.

9. What E1 to E3 found (added 12 October 2026, evening)

The three free experiments ran as separate sessions after Section 6 was written, each with a pre-registration file written before computing, each reproducible by one script, none with model calls. Their full reports are E1-RESULTS.md, E2-RESULTS.md and E3-RESULTS.md next to this file. This section states what each found against the hypotheses of Section 5 and what in Sections 1 to 8 must change.

9.1 E1: R2E-Gym finished and bootstrapped

Numbers of Section 4. Every value of Tables 4.1 to 4.3 lies inside its 95% bootstrap interval over bugs (1,000 resamples). Nothing to correct.

H1 is wrong as written. The pre-registered statistic was share(16) − share(4). For the argmax rule it is −0.08 [−0.12, −0.04] and for threshold 0.5 it is −0.08 [−0.11, −0.06]: the share falls, as H1 says. For threshold 0.6 it is +0.03 [−0.01, +0.06] and for threshold 0.7 it is +0.04 [+0.01, +0.07]: flat and rising, up to n=25. At those thresholds β is 0.49 and 0.76, few candidates are accepted, and adding candidates mostly adds accepted correct ones; consequence (b) of Section 3.2 predicts exactly this, and the pooled formula with the measured (α, β) reproduces the rise at 0.7. H1 must read: the share falls with n once the healing of false rejects is exhausted, which at β above 0.5 is beyond n=25 on this data. A related fact: the best threshold rises with n, from 0.35 at n=2 to 0.6 at n=16 and 25.

H3 holds, and the open question of Section 4.4 is closed. ICC of verdicts within a bug is at least 0.26 at every threshold (lowest interval bound 0.17). At thresholds 0.6 and 0.7 the pooled formula overpredicts by 3.8 and 5.7 points at n=12 on the held-out half and split-half per-bug rates remove 0.36 and 0.65 of that error; at 0.5 the pooled formula is already within 1.3 points and the clause does not apply. The new part: when verdicts are regenerated as independent coins from the in-sample per-bug rates, so that the per-bug model is true by construction, the split-half removal is the same as on the real verdicts within 0.08 at every $n \geq 4$. The error that split-half does not remove is therefore estimation noise from about 13 candidates per half, not structure the per-task model misses, within the resolution of this test.

H4 holds on an independent difficulty measure. Against the paper's item-response difficulty per bug (fitted on 175 other systems; 474 of 500 bugs matched), Spearman correlation of β_i is +0.46 [+0.36, +0.55] and of α_i +0.09 [−0.01, +0.18] at threshold 0.6; against $1 - p_i$, +0.53 and +0.06. By difficulty tercile β_i is 0.47, 0.72, 0.75 and α_i 0.16, 0.13, 0.14.

H2, formula cells. On the R2E k_i the strict point (α 0.05, β 0.6) collects 0.45, 0.52, 0.54 of the prize at n = 4, 8, 16 and the lenient point (α 0.2, β 0.1) collects 0.64, 0.55, 0.46. The crossing lies between n=8 and 16, so H2's formula clause should read "by n=16" on this distribution.

H6 cannot be decided without a value V per solved task, which no paper gives. As a break-even: the marginal gain of the model verifier (argmax) is 0.42 points of solved tasks per added candidate from 4 to 8 and 0.19 from 8 to 16; at $1 per rollout and verification at 6% of it, stopping at n=8 is right when a solved task is worth between about $255 and $550; the verifier's price hardly matters, the rollout price and V decide.

9.2 E2: Terminal-Bench, no judge scores to measure

The repository of the LLM-as-a-Verifier paper (2607.05391) ships the harness reward of every trajectory (Terminal-Bench 2.1, 89 tasks × 5 trials of one model; Terminal-Bench 4.0, 66 × 5; Terminal-Bench 2.0, 89 × 5; SWE-bench Verified, 3 models × 500), but not the judge's scores: the score caches are git-ignored and absent from every branch and tag, and the judge is pairwise, so even a recovered cache would need a win-share conversion. Section 6's description of E2, taken from a literature pass, was wrong on this point. Rewards recounted from the files reproduce the paper's README exactly (pass@1 / oracle: 0.787 / 0.966, 0.648 / 0.788, 0.831 / 0.921, 0.761 / 0.844). H1, H3 and H4 are not testable there at $0. What E2 adds: the authors' aggregate selector shares, 0.52 (self-verification, TB2.1), 0.38, 0.31 and 0.25 (0.18 against the best member), all inside the literature range of Section 2.2; and the formula's prediction on these strong pools that a difficulty-tracking α costs much more where the contested tasks are the few hard ones (TB2.1: 0.71 → 0.39 at α 0.2, n=4) than where difficulty is spread (TB4.0: 0.73 → 0.62), the top-seven pattern of Section 3.3 again.

9.3 E3: the agent's own tests are not a strict verifier

Data: 67,074 OpenHands rollouts of Qwen3-Coder-480B on 3,792 issues (6,306 with at least one candidate; median 10 per issue), the benchmark's resolved flag on every row, and on 47% of rows two 0/1 columns: whether the agent's own generated tests pass its own patch (V1) and whether those tests are valid, that is fail on the original code and pass on the reference fix (used only in the upper-bound verifier V2 = V1 and valid). Also V3 = the agent submitted a patch. Reading the needed columns took 3 MB of disk and 91 seconds. Pool: pass@1 0.476, pass@n 0.601, prize 12.6 points, a third of R2E's; 1,741 contested issues at median base rate 0.57.

The expected strict verifier is not here. V1 has α 0.69, β 0.24, AUROC 0.53: the agent's tests pass its own patch on 72% of rows whether the patch is right or not. Only 30% of the agent's test sets are valid against the reference fix, and even on those a wrong patch passes 72% of the time. V1 collects 0.02 of the prize at n=2 and 0.04 at n=8, with intervals touching zero. V3, submission, has α 0.85 and β 0.04. So on this agent the tests it writes for itself sit in group (a) of Section 1, lenient, not in group (b); the most likely reason is that the same model wrote patch and tests to the same misreading of the issue. The only execution-free signal with information is trajectory length (fewer assistant turns: AUROC 0.68, share 0.10 at n=2 falling to 0.05 at n=8), as on R2E.

H4 accept side is falsified for this verifier: α_i against p_i is +0.03 [−0.01, +0.07], and α_i is about 0.70 in every difficulty bin. The test verifier is lenient everywhere, so there is no difficulty structure on the accept side to find. ICC within issue is 0.47 among wrong and 0.48 among right candidates, higher than R2E's 0.26 to 0.43: the agent's tests pass or fail its rollouts on an issue together.

H2 is not falsified in letter and not testable in substance with V1: share(8) − share(4) is −0.01 [−0.03, +0.01] on the 1,306 issues with eight tested candidates, but a verifier collecting 0.05 has nothing to lose. The asymmetry does show between verifiers. The lenient V3 loses share with n: −0.015 [−0.026, −0.004] from n=4 to 8 on the same 4,737 issues, the H1 shape. The strict V2 (α 0.12, β 0.63) gains: +0.032 [+0.007, +0.057] on the same 1,306 issues, 0.22 → 0.25, the H2 shape. Both intervals exclude zero, but V2 is built with the reference fix, so this is the predicted shape on real k_i, not a deployable verifier showing it.

H3 holds once the verifier has teeth. For V2 the pooled formula overpredicts by 1.4 points at n=4 and 2.0 at n=8, in-sample per-issue rates reproduce the observation, and split-half rates remove 0.51 and 0.64 of the pooled error at n=2 and 4 with two or three candidates per half. For V1 the pooled formula fits within half a point, which is not a success of i.i.d. verdicts but a consequence of α 0.69: when nearly everything is accepted the threshold rule is a random pick and any formula predicts pass@1.

Missingness check: rows without tests resolve 4 points more often than rows with tests, but within issues the difference is +0.002, so it is between issues; rows without tests are shorter and more often submitted. Hybrids of V1 with a second signal collect at most 0.13, about what the second signal collects alone.

9.4 What changes in the document and in the plan

- **H1** is restated: the share falls with n for argmax rules and for threshold rules with β below about 0.5; at larger β it rises until the healing of false rejects is exhausted, beyond n=25 on R2E. The optimal threshold rises with n.
- **H2's formula clause** reads "by n=16". Its data clause is untested: no free data set contains a deployable strict test verifier. The agent's own tests are lenient (E3); validated tests need the reference fix; the Terminal-Bench judge scores are not released.
- **H3** is supported on two data sets and three verifiers with non-trivial error; the remaining split-half error on R2E is estimation noise. The qualifier "where the verifier has teeth" is added: a verifier with α near 0.7 gives the formula nothing to get wrong.
- **H4**: the reject side holds for the model verifier on two difficulty measures; the accept side is falsified for self-written tests, whose α is flat in difficulty. The claim that test-based verifiers have their blind spot on the accept side is withdrawn for self-written tests and remains open for tests from another model (E4, E5).
- **Section 1, group (b)**: "tests the agent wrote" is moved from the strict group to the lenient group when the same model wrote the patch; the strict examples that stand are reference-validated tests, formal checkers and exact match.

- **Section 6**: E2's data description is corrected (no judge scores shipped). E4 (five o1 rollouts with cross-run generated tests, correctness recomputed with the SWE-bench harness) is promoted from "only if E3 falls short" to the next step, because it is now the only free source of a per-candidate test verifier not written by the solver itself. E5 (our own judge and a test-writer from another family on the same candidates, $20 to $50) remains the only way to test H5 and the one way to get a strict test verifier without the reference fix.
- **H6** stays a break-even statement until a value per solved task is chosen.

9.5 Verifier independence, measured before any verifier is run

H5 says a verifier whose errors are not aligned with the solver's collects more at equal (α, β). Whether two model families have unaligned errors can be measured from existing correctness matrices before any verifier is built: fit the one-difficulty-factor model of Section 11.2 of the paper (two-parameter item response), take each system's residual on each bug (solved or not, minus the model's prediction), and correlate residuals between systems. On the SWE-bench Verified matrix, 91 submissions with a known model family and accuracy at least 0.30 (Claude 40, OpenAI 18, Gemini 7, Qwen 6, Kimi 6, Mistral 4, GLM 3, two or one each for MiniMax, DeepSeek, Llama, Amazon, Doubao), the family-pair means with 95% bootstrap intervals over the 474 fitted bugs are (working files swarm-lbd/experiments/partial-verifier/verifier-independence/):

Table 9.1. Mean residual correlation between systems of two families after the difficulty factor. Diagonal: different systems of the same family. Positive: shared errors beyond difficulty; negative: fewer than chance.

Solver family	Least aligned verifier families	Most aligned	Within family
Qwen (the SWE-rebench rollouts of E3 and E5)	Claude −0.018 [−0.03, −0.01], DeepSeek −0.020 (2 systems, wide), OpenAI −0.011	Amazon +0.052, Mistral +0.019, Gemini +0.004	+0.092 [+0.06, +0.13]
OpenAI (the o1 rollouts of E4)	Doubao −0.029, Claude −0.019, Kimi −0.019, Qwen −0.011	DeepSeek +0.035, Gemini +0.030, GLM +0.018	+0.035 [+0.02, +0.05]
Claude	almost every family −0.01 to −0.03	MiniMax +0.002, Kimi 0.000	+0.008 [0.00, +0.01] (40 systems, many scaffolds)

The same question on the paper's text corpus, as a first look. Before the SWE-bench matrix, the same idea was tried on the paper's seven-model text corpus (six models after a missingness filter, 733 problems, 358 contested), with the simpler additive residual, correctness minus the problem's mean over models, in place of the item-response fit. Raw φ over all problems: Haiku with DeepSeek 0.58, with Sonnet 0.55, with Qwen 0.54, with Mistral 0.43, with Opus 0.44; Opus with

Sonnet 0.58, with Mistral 0.32. Residual correlation on contested problems, where every value is negative by construction (six residuals per problem sum to zero, expected mean about −0.2) and must be read relatively: Haiku with DeepSeek −0.11, Sonnet −0.14, Qwen −0.16, Opus −0.28, Mistral −0.36; Opus with Sonnet −0.03; Mistral with everyone −0.31 to −0.36. So on text a Haiku solver's errors are least shared with Mistral and Opus and most with DeepSeek, more than with Sonnet of the same vendor: family by name and family by errors are not the same thing. Mistral is the most independent of all and also the weakest (0.79), which is the formula's trade-off between error independence and error rate. Opus and Sonnet are the most "family" pair. This is additive, on text, on six models with wide intervals; the SWE-bench table above is the version that matters for agents.

Two readings. First, after the difficulty factor every residual is small, $|r| \leq 0.09$: the paper's finding that one factor explains almost all co-failure on SWE-bench holds at the family level too. Choosing the verifier's family therefore moves its alignment with the solver by a few hundredths, far less than the difficulty-tracking α of Section 3.3, which moved the share from 0.67 to 0.32. The independence lever H5 exists on this benchmark but is small, and E5 is sized to bound it rather than to find it (Section 9.7). Second, the table still says which family to use: for Qwen rollouts a Claude or OpenAI verifier, not Qwen and not Mistral; for o1 rollouts Claude, Kimi or Qwen, not DeepSeek or Gemini. Caveats: "family" mixes scaffolds, versions and dates; several families have one or two systems; intervals are over bugs, not over systems; the text-corpus look above is on a different scale and is not comparable.

9.6 E4: the submission does not contain the rollouts' patches

The W&B Programmer o1 submission (20250117, 5 rollouts per bug, cross-run generated tests) was downloaded in full (506 MB) and read. It holds the five trajectories and the cross-check matrix but only the one submitted patch per bug; the other four patches were never saved. Rebuilding them by replaying the trajectories' editor calls against the repository was tried on the 500 submitted rollouts, where the published patch is the reference: 277 exact, 66 equal up to whitespace, 45 a subset of the published patch, 111 different, 1 without a reference; 176 of the 2,500 rollouts run file-writing shell commands the replay cannot emulate. At 69 to 78% fidelity, labels from the harness on rebuilt patches would carry a systematic error (a wrong rebuild fails the tests), so no evaluation was submitted and E4 is closed without labels. Two facts from the data remain. The submitter scored all five rollouts with the hidden tests and ran the cross-check only on the 156 bugs where they disagreed, submitting the first rollout elsewhere; the leaderboard number is therefore not a selector's pass@1, which bears on the 82 selector-flag submissions of the paper. And the cross-check verdicts are attributable to their source rollout, so a per-candidate cross-run test matrix exists if the patches are ever released; a request to the submitter is the one remaining path.

9.7 E5 as pre-registered

E5 (swarm-lbd/experiments/partial-verifier/E5/PREREG.md) puts three verifiers on the same 1,600 candidates: a Claude Haiku judge, tests written by Claude Haiku, and tests written by a Qwen model, executed in cloud containers on 200 contested SWE-rebench issues with 8 Qwen3-Coder rollouts each. Contested issues are sampled on purpose (the prize is zero elsewhere; α and β are candidate-level and unaffected), and the primary endpoints are α and β per verifier with a cluster bootstrap over issues, the share being secondary. Power, simulated on the real candidate counts with per-issue rates at ICC 0.4: 0.8 for an α difference of 0.10 and a share difference of 0.13 at n=8 (the sizes H2 predicts between test and judge), 0.35 to 0.45 for an α difference of 0.05 (the size Table 9.1 predicts for H5), 1.0 for a β difference of 0.30. H5 is therefore pre-registered as a bounding measurement: an interval within ±0.05 reads "not distinguishable from zero at this size", and detecting 0.05 would need 600 to 800 issues. Cost: under $20 of model calls (the Claude calls run under a subscription), $5 to $30 of cloud execution for about 4,000 test runs.

9.8 E5: what was found (run the same evening, PREREG unchanged)

E5 ran as pre-registered: 200 contested SWE-rebench issues with a public image, 8 Qwen3-Coder rollouts each (pass@1 0.536, pass@8 0.960, prize 42.4 points on these issues), three verifiers on the same 1,600 candidates, 2,000 cloud sandboxes, all completed. Full report E5-RESULTS.md; spend: OpenRouter $0.11, Modal about $2.7 by sandbox-seconds, 1,800 Claude Haiku calls under the subscription.

Table 9.2. E5 verifiers against hidden tests (95% cluster-bootstrap intervals over issues). Share = share of the prize on the sampled contested issues at the threshold rule unless marked argmax.

Verifier	α	β	AUROC	Share, n=2	n=4	n=8
Judge, Claude Haiku, threshold 0.5	0.20 [0.15, 0.26]	0.54 [0.48, 0.60]	0.68	0.17	0.15	0.15 [0.08, 0.22]
Judge, argmax				0.32 [0.22, 0.41]	0.23	0.19 [0.07, 0.31]
Tests written by Claude Haiku, strict (all pass)	0.39 [0.31, 0.47]	0.48 [0.40, 0.55]	0.57	0.15	0.12	0.11 [0.06, 0.17]
Tests written by Qwen3-Coder-30B, strict	0.11 [0.06, 0.16]	0.86 [0.80, 0.91]	0.50	0.06	0.05	0.04 [0.01, 0.08]

Claude tests, only those valid against the reference fix (upper bound)	0.29 [0.22, 0.36]	0.49 [0.41, 0.57]		0.19	0.15	0.14 [0.09, 0.19]
Strict Claude tests, then judge argmax				0.35	0.26	0.22 [0.09, 0.35]

Against the pre-registered endpoints. P1 (tests stricter than the judge) is falsified: α of the Claude-written tests is 0.39 against the judge's 0.20, difference +0.18 [+0.09, +0.27], with no higher false-reject rate. P2 is undecided: share difference −0.04 [−0.11, +0.03], and neither share moves between n=4 and 8. P3 (family) is falsified by its letter but not at matched operating points: the Qwen tests reject 0.86 of correct patches and carry no information (AUROC 0.50), so their low α is that of a verifier that rejects nearly everything; the correlation clause is flat (−0.02 [−0.21, +0.18]). Nothing can be said about family independence from this pair, as Section 9.5 led us to expect. P4 (tests' blind spot on the accept side) is falsified again: +0.12 [−0.01, +0.27]. P5 (per-task rates) holds for the judge at every threshold: ICC 0.65 to 0.70, the pooled formula overpredicts by 6.7 to 8.3 points at n=8, split-half rates remove 0.39 to 0.46 of the error; and the judge repeats the R2E pattern, a blind spot on the reject side (β_i against difficulty +0.39 [+0.26, +0.51]) and a flat α_i (+0.08).

Why the tests failed, and what it does and does not show. By design the test writers saw only the issue text and the list of file names, never file contents, so that no hidden information could enter. The result is tests that mostly test the wrong thing: on only 98 of 200 issues does at least one Claude-written test fail on the base commit and pass on the reference fix (40 of 200 for Qwen), and on 98 issues every Claude test passes without the fix. A wrong patch passes such tests about as often as a right one. So E5 shows that a blind test writer is a lenient verifier, not that execution-based verifiers are; H2 for a test writer that reads the repository, as the CodeMonkeys and R2E test agents do, remains untested, and that is now the one experiment the line still needs (E5b: an agentic test writer with repository access, same candidates, same budget). Two further readings. The judge used as a ranker collects the most, 0.32 at two candidates falling to 0.19 at eight, the H1 shape once more; gating by strict tests first adds nothing beyond the judge's interval. And the per-issue clustering of test verdicts is extreme (ICC 0.89 to 0.99: a written test file passes or fails all candidates of an issue together), which is why the pooled formula fits the test verifiers within a point: a verdict that hardly depends on the candidate gives the formula nothing to get wrong, as with the solver's own tests in E3.

Execution notes. 328 of the first-pass candidate sandboxes failed to apply the patch (model_patch strings without a final newline; the agent's scratch file already present in the image); the second pass fixed both and all 1,599 non-empty patches applied. The 122 contested

issues without a public image are outside the frame. The whole-benchmark share is the share on contested issues times 0.29.

9.9 E5b: a test writer that reads the repository (13 October)

E5b repeats E5 with one change, pre-registered in E5b-PREREG.md before the run: the Claude Haiku test writer gets read-only tools (Read, Grep, Glob) on a clone of the repository at the base commit, so it can find the code the issue is about; it still never sees a patch, the reference fix or the hidden tests. Same 200 issues, same 1,600 candidates, same judge scores, same execution; E5's blind Claude tests were re-run in the same sandboxes as the within-run control. Median 8 agent turns per issue, 200 subscription calls, 2,000 sandboxes, about $2.2 of Modal. Full report E5b-RESULTS.md.

Table 9.3. E5b: agentic tests against the blind tests of E5 and the judge, on the same candidates. Share = share of the prize on the sampled contested issues.

Verifier	Issues with a reference-valid test	α	β	AUROC	Share *n*=2	*n*=4	*n*=8
Agentic Claude tests, strict (T_A)	163 of 200	0.46 [0.39, 0.53]	0.25 [0.19, 0.31]	0.65	0.34 [0.27, 0.42]	0.28	0.23 [0.16, 0.30]
Blind Claude tests, strict (E5)	98 of 200	0.39 [0.31, 0.47]	0.48 [0.40, 0.55]	0.57	0.15	0.12	0.11
Judge, threshold 0.5		0.20	0.54	0.68	0.17	0.15	0.15
Judge, argmax					0.32	0.23	0.19
Agentic tests as gate, then judge argmax					0.42 [0.31, 0.52]	0.32	0.28 [0.15, 0.40]
Agentic tests, reference-valid only (upper bound)		0.45	0.20		0.38	0.31	0.26

Reading. Repository access changed what the tests are, not how lenient they are. The agent's tests are valid against the reference fix on 163 issues instead of 98, the false-reject rate fell from 0.48 to 0.25 and the AUROC rose from 0.57 to 0.65; but the false-accept rate did not fall, 0.39 → 0.46 [0.39, 0.53]. The tests check the behaviour the issue names, and a patch the hidden tests reject usually does fix that behaviour and fails elsewhere: regressions, edge cases, other test files.

So the pre-registered claim that tests are stricter than the judge (P1') is falsified a second time, now with a test writer that reads the code (α difference +0.26 [+0.17, +0.34]), and the claim that access lowers α by 0.10 (P6) is falsified by its letter (+0.07 [−0.02, +0.16]) while its coverage clause holds. What the agentic tests do collect is real and the largest in this line: 0.34 of the prize at two candidates and 0.23 at eight, against the judge's 0.17 and 0.15 (P2': +0.08 [0.00, +0.16] at n=8, at the edge), and the gate-then-rank hybrid reaches 0.42 at two and 0.28 at eight. The deployable strict rule sits within 0.04 of its reference-valid upper bound, so validity is no longer the limit; the tests' discrimination is. The accept-side blind spot appears for the first time, weakly: α_i against difficulty +0.26 [+0.13, +0.40], α by tercile 0.28 / 0.34 / 0.56 from easy to hard (P4' neither met nor falsified). H3 holds on ICC (0.79 / 0.91) and split-half removal (0.37); at α 0.46 the pooled formula is within a point and has nothing to get wrong.

What this settles. Across E3, E5 and E5b, three ways of getting execution-based tests on agent patches (the solver's own, a blind writer, a reading writer) all have false-accept rates between 0.39 and 0.69, while the one strict execution-based verifier in the data, reference-validated tests in E3 (α 0.12), needs the hidden fix. H2's expectation that execution-based verifiers sit in group (b) of Section 1 was wrong for model-written tests on this benchmark: their limit is α, set by how often a wrong patch fixes the literal symptom, which on these issues is close to half. The formula's conclusion stands in a sharper form: the prize is collected by a verifier with small α, and model-written tests do not supply one; they supply recall. The best use found for them is as a gate in front of a ranking judge.

One sentence for the paper: a verifier's two error rates, measured per task, predict what a selector collects; on agent tasks the prize goes to a verifier with a small false-accept rate, and neither model judges nor model-written tests, blind or reading the code, supply one, so the best deployable rule found collects 0.28 of the prize at eight candidates by gating on tests and ranking by a judge.

10. Hypotheses against experiments: the ledger

Sections 9.1 to 9.9 report by experiment. This section reads the same results by hypothesis. "Holds" and "falsified" refer to the pre-registered thresholds of Section 5 and the E5/E5b pre-registrations; "revised" means the hypothesis survives with a qualifier it did not have.

Table 10.1. What each experiment said about each hypothesis. R2E = E1 (released model verifier); TB = E2 (Terminal-Bench, no judge scores); own = E3 (agent's own tests); o1 = E4 (no per-rollout patches); E5 = judge, blind tests; E5b = reading tests.

Hypothesis	E1 R2E	E2 TB	E3 own tests	E4 o1	E5 judge / blind tests	E5b reading	Verdict

						tests	
H1 precision ceiling: share falls with *n*	argmax −0.08 [−0.12, −0.04] (falls); threshold 0.7 +0.04 [+0.01, +0.07] (rises)	not testable	own tests collect 0.02–0.04, nothing to fall; "submitted" falls −0.015 [−0.026, −0.004]	no labels	judge argmax 0.32 → 0.19 (falls); threshold 0.5 flat 0.17 → 0.15	reading tests 0.34 → 0.23 (falls, −0.05 [−0.07, −0.03] from 4 to 8)	**Revised:** the share falls with *n* for argmax and lenient thresholds; at β above 0.5 it rises until the healing of false rejects is exhausted (beyond *n*=25 on R2E).
H2 asymmetry: execution-based verifiers strict (small α), gain with *n*; model verifiers lenient	formula cells: strict point overtakes lenient between *n*=8 and 16	not testable	own tests α 0.69, not strict; reference-validated V2 (α 0.12) gains +0.032 [+0.007, +0.057]	no labels	blind tests α 0.39 vs judge 0.20: falsified (P1)	reading tests α 0.46 vs judge 0.20: falsified again (P1'); access did not lower α (P6, +0.07 [−0.02, +0.16])	**Falsified for model-written tests:** on this benchmark they are lenient (α 0.39–0.69) and supply recall; the asymmetry itself shows only with the hidden fix (V2) and in the formula.
H3 per-task rates: ICC > 0.2, pooled formula overpredicts, split-half removes > 1/3	ICC 0.26–0.43; overprediction 3.8 / 5.7 pts; removal 0.36 / 0.65; remainder = estimation	not testable	ICC 0.47–0.51; for V1 pooled fits (α 0.69 leaves nothing to miss); for	no labels	judge ICC 0.65–0.70, overprediction 6.7–8.3 pts, removal 0.39–0.46: holds	reading tests ICC 0.79 / 0.91, removal 0.37; pooled within a	**Holds** wherever the verifier has information (three verifiers, two data

	noise (synthetic within 0.08)		V2 removal 0.51 / 0.64			point at α 0.46	sets); vacuous where α is near 0.5 or above.
H4 blind spot: model verifiers on the reject side, test verifiers on the accept side	β_i ~ IRT difficulty +0.46 [+0.36, +0.55], α_i +0.09: reject side holds	not testable	own tests: α_i +0.03, flat everywhere: accept side falsified	no labels	judge β_i +0.39 [+0.26, +0.51], α_i +0.08: reject side holds; blind tests α_i +0.12 [−0.01, +0.27]: accept side falsified	reading tests α_i +0.26 [+0.13, +0.40], α by tercile 0.28 / 0.34 / 0.56: accept side weakly present, below the 0.3 threshold	**Reject side holds** for both model verifiers; **accept side** absent for self-written and blind tests, weakly present for reading tests.
H5 independence: a verifier from an error-independent family collects more at equal (α, β)	—	—	—	—	Qwen tests reject nearly everything (AUROC 0.50), no matched comparison (P3 not decidable)	both writers Claude; no family contrast	**Untested on data; bounded by Section 9.5:** residual family correlations on SWE-bench ≤ 0.09, so the lever is small; the formula's version (Section 3.3) stands as prediction.
H6 cost: money-optimal *n*	marginal gain 0.42 pts per candidate	—	—	—	Modal $4.47 for 4,000 sandboxes,	same	**Undecided:** no published

between 4 and 8	from 4 to 8, 0.19 from 8 to 16; break-even value of a solved task $255–550 at $1 per rollout for stopping at 8				OpenRouter $0.11, Claude under subscription		value per solved task; stated as a break-even. The verifier's price is small against the rollout's.
E6 write-up	Done 12–13 October: paper §2.2 replaced by the per-task formula (v9), §7.7 Table 7.5 (released verifier, v9) and Table 7.6 (own runs, v10), §3 ledger row, §12, §14; note "What an Unreliable Verifier Collects" published 13 October (no DOI yet).						Done.

In one line: the per-task two-rate formula held on every verifier with information; the model verifiers' task-dependence sits on the reject side; model-written tests are lenient on this benchmark and supply recall, not a small α; the strict verifier the formula asks for exists in the data only through the hidden fix; and the best rule found, tests as a gate before a ranking judge, is a lead with an overlapping interval, not a result.

Author's working files: swarm-lbd/experiments/partial-verifier (FORMALISM.md, LIT-agents.md, LIT-theory.md, LIT-datasets.md, sim_select.py, r2e_analysis.py, r2e_model_check.py, r2e_splithalf.py). Nothing in this document is published or committed.